

PowerShell — Travaux pratiques

Formation : Administration Microsoft 365 — Module PowerShell

Durée indicative : 3 à 4 h

Prérequis : PowerShell 5.1 ou PowerShell 7+ installé, politique d'exécution `RemoteSigned` (scope `CurrentUser`)

Support de cours : *PowerShell — Syntaxe et instructions courantes* (fourni en PDF)

Consignes générales

RÈGLE	DÉTAIL
Environnement	Console PowerShell (<code>pwsh</code> ou <code>powershell</code>)
Livrables	Scripts <code>.ps1</code> numérotés ou commandes saisies dans un fichier texte
Nommage	<code>exo-niveauX-Y.ps1</code> (ex. <code>exo-niveau1-1.ps1</code>)
Aide	<code>Get-Help</code> , <code>Get-Command</code> , support de cours
Sécurité	Ne pas exécuter de scripts non vérifiés téléchargés

Astuce

Testez chaque exercice ligne par ligne avant de regrouper dans un script. Utilisez `Write-Host` avec `-ForegroundColor` pour distinguer les sorties.

Niveau 1 — Bases

Objectif : prendre en main la console, les variables et les cmdlets fondamentales.

Exercice 1.1 — Affichage

Afficher `Bonjour` sur une ligne, puis ton prénom sur la ligne suivante, avec `Write-Host` .

Critère de réussite : deux lignes distinctes s'affichent dans la console.

Exercice 1.2 — Variable

Stocker ton âge dans une variable, puis afficher le message : `J'ai X ans` (X = valeur de la variable).

Critère de réussite : la variable est réutilisée dans une chaîne interpolée.

Exercice 1.3 — Fichiers du dossier courant

Lister les fichiers et dossiers du répertoire courant avec `Get-ChildItem`.

Critère de réussite : la commande affiche le contenu de `$PWD` sans erreur.

Exercice 1.4 — Date et heure

Afficher la date et l'heure actuelles.

Critère de réussite : utilisation de `Get-Date` (format libre ou formaté).

Niveau 2 — Variables et types

Objectif : manipuler chaînes, nombres, tableaux et conversions de type.

Exercice 2.1 — Carré d'un nombre

Demander un nombre à l'utilisateur avec `Read-Host`, le convertir en entier, puis afficher son carré.

Critère de réussite : saisie `5` → affichage `25`.

Exercice 2.2 — Concaténation

Créer deux variables chaînes (prénom et nom), les concaténer et afficher le résultat complet.

Critère de réussite : une seule chaîne affichée, sans espace manquant entre les parties (si souhaité).

Exercice 2.3 — Tableau de prénoms

Créer un tableau de 5 prénoms et afficher uniquement le 3^e élément (index PowerShell).

Critère de réussite : le 3^e prénom s'affiche (pas le 1^{er} ni le 5^e).

Exercice 2.4 — Conversion de type

Stocker le nombre `42` dans une variable chaîne, le convertir en entier avec `[int]`, puis l'utiliser dans un calcul (ex. `+ 8`) et afficher le résultat.

Critère de réussite : résultat numérique `50`, pas de concaténation de chaînes.

Niveau 3 — Structures de contrôle

Objectif : utiliser conditions, boucles et `switch`.

Exercice 3.1 — Pair ou impair

Écrire un script qui demande un nombre à l'utilisateur et affiche s'il est **pair** ou **impair** (`if` / `else`).

Critère de réussite : `4` → `pair` ; `7` → `impair`.

Exercice 3.2 — Boucle for

Boucler de **1 à 10** avec `for` et afficher chaque nombre sur une ligne.

Critère de réussite : dix lignes numérotées de 1 à 10.

Exercice 3.3 — Fichiers .txt

Créer un tableau contenant des noms de fichiers fictifs (ex. `rapport.txt`, `data.csv`, `notes.txt`, `image.png`). Parcourir le tableau avec `foreach` et afficher **uniquement** ceux qui se terminent par `.txt` (`if` dans la boucle).

Critère de réussite : seuls les fichiers `.txt` apparaissent.

Exercice 3.4 — Switch jour de la semaine

Demander un numéro de 1 à 7 à l'utilisateur et afficher le nom du jour correspondant avec `switch` (1 = Lundi, 2 = Mardi, ..., 7 = Dimanche).

Critère de réussite : `3` → `Mercredi` ; valeur hors plage → message d'erreur ou `default`.

Niveau 4 — Fonctions

Objectif : encapsuler la logique dans des fonctions réutilisables.

Exercice 4.1 — Get-Carre

Créer une fonction `Get-Carre` qui prend un paramètre `[int]$Nombre` et **retourne** son carré (utiliser `return`).

Critère de réussite : `Get-Carre -Nombre 6` retourne `36`.

Exercice 4.2 — Paramètre optionnel

Créer une fonction `Get-Salutation` avec un paramètre obligatoire `$Nom` et un paramètre optionnel `$Titre` (valeur par défaut : `"M."`). La fonction affiche : `Bonjour M. Dupont` ou `Bonjour Dr. Martin` si le titre est fourni.

Critère de réussite : appel sans `-Titre` utilise la valeur par défaut.

Exercice 4.3 — Test-Path

Créer une fonction `Test-FichierExiste` qui prend un chemin de fichier et retourne `$true` ou `$false` selon que le fichier existe (`Test-Path`).

Critère de réussite : chemin existant → `$true` ; chemin inexistant → `$false` .

Niveau 5 — Pipeline et objets

Objectif : filtrer, trier et exporter des objets via le pipeline.

Exercice 5.1 — Processus gourmands

Lister les processus consommant **plus de 100 Mo** de RAM avec `Get-Process | Where-Object` .

***Indice** : la propriété mémoire s'appelle `WorkingSet` (en octets). 100 Mo ≈ `100MB` en PowerShell.*

Critère de réussite : tableau filtré avec au moins les colonnes `Name` et mémoire.

Exercice 5.2 — Tri par taille

Dans un dossier de ton choix, lister les **fichiers** (pas les dossiers), les trier par taille décroissante (`Sort-Object`) et afficher les 10 plus gros.

Critère de réussite : ordre décroissant vérifié sur la propriété `Length` .

Exercice 5.3 — Export CSV des services

Exporter la liste des services Windows (`Get-Service`) vers un fichier `services.csv` avec `Export-Csv` (sans ligne de type PowerShell : `-NoTypeInfo`).

Critère de réussite : fichier CSV ouvrable dans Excel ou Notepad, colonnes `Name` , `Status` , `DisplayName` visibles.

Niveau 6 — Scripts et gestion d'erreurs

Objectif : structurer un script complet avec gestion d'erreurs.

Exercice 6.1 — Sauvegarde de dossier

Écrire un script `.ps1` qui copie récursivement un dossier source vers un dossier de destination avec `Copy-Item -Recurse`.

Critère de réussite : après exécution, les fichiers source sont présents dans la destination.

Exercice 6.2 — Try / Catch

Reprenant l'exercice 6.1 (ou une lecture de fichier), entourer l'opération d'un bloc `try` / `catch` qui affiche un message clair si le chemin source est introuvable.

Critère de réussite : chemin invalide → message d'erreur personnalisé, pas de pile rouge brute (ou `$ErrorActionPreference` adapté).

Exercice 6.3 — Module simple

Créer un module PowerShell (fichier `.psm1`) contenant **2 fonctions** réutilisables (ex. `Get-Carre` et `Test-FichierExiste` des niveaux précédents). L'importer avec `Import-Module` et appeler les deux fonctions.

Critère de réussite : `Get-Module` affiche le module chargé ; les deux fonctions sont invoquables.

Exercice 7 — Gestionnaire de fichiers (projet intégrateur)

Contexte : créer un script qui organise un dossier de fichiers en les triant par extension.

Étape 1 — Préparer l'environnement

Créer un dossier `C:\TestFiles` contenant quelques fichiers de test (`.txt`, `.csv`, `.log`).

Étape 2 — Script de tri

Écrire un script qui :

1. Liste tous les **fichiers** du dossier `C:\TestFiles` (pas les sous-dossiers créés ensuite)
2. Pour chaque extension trouvée (`.txt` , `.csv` , `.log` ...), crée un sous-dossier portant le nom de l'extension **sans le point** (ex. `TEXT` , `CSV` , `LOG`)
3. Déplace chaque fichier dans le sous-dossier correspondant

Critères de réussite :

VÉRIFICATION	ATTENDU
Structure	<code>C:\TestFiles\TEXT\fichier.txt</code>
Extensions multiples	chaque type dans son dossier
Dossier source	plus de fichiers à la racine (seulement des sous-dossiers)

Étape 3 — Résumé

À la fin du script, afficher un résumé : **nombre de fichiers déplacés par type** (ex. `TEXT : 3` , `CSV : 2`).

Critère de réussite : totaux cohérents avec les fichiers initialement créés.

Bonus 7.4 — Éviter l'écrasement

Ajouter un `try` / `catch` (ou logique équivalente) pour gérer le cas où un fichier existe déjà dans le dossier de destination : **ne pas écraser** — renommer avec un suffixe `_1` , `_2` , etc.

Critère de réussite : relancer le script deux fois sans perte ni écrasement de fichier.

Bonus 7.5 — Paramètre de chemin

Ajouter un paramètre au script pour que le chemin source ne soit pas codé en dur :

```
param(  
    [Parameter(Mandatory)]  
    [string]$CheminSource  
)
```

Critère de réussite : `.\Organiser-Fichiers.ps1 -CheminSource "D:\AutreDossier"` fonctionne.

Grille d'évaluation (indicative)

NIVEAU	EXERCICES	POINTS
1 — Bases	1.1 à 1.4	8
2 — Variables	2.1 à 2.4	8
3 — Contrôle	3.1 à 3.4	12
4 — Fonctions	4.1 à 4.3	12
5 — Pipeline	5.1 à 5.3	12
6 — Scripts	6.1 à 6.3	15
7 — Projet	7.1 à 7.3	20
Bonus 7.4 + 7.5		+5 chacun
Total		87 (+10 bonus)

Rendu

Remettre :

1. Un dossier `TP-PowerShell-{Nom}` contenant les scripts `.ps1`
2. Le fichier `services.csv` (exercice 5.3)
3. Une capture d'écran du résumé de l'exercice 7

Formateur : Rayan Mechety — RedStone Formations